

KNeighborsClassifier#

[https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.ht](https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html)

Description:

Classifier implementing the k-nearest neighbors vote. Read more in the User Guide. Number of neighbors to use by default for kneighbors queries. Weight function used in prediction. Possible values:

Function Signature

```
class sklearn.neighbors.KNeighborsClassifier(n_neighbors=5,  
*, weights='uniform', algorithm='auto', leaf_size=30, p=2,  
metric='minkowski', metric_params=None, n_jobs=None)  
[source]#
```

Parameters:

`n_neighbors`int, default=5

`weights`{'uniform', 'distance'}, callable or None, default='uniform'

`algorithm`{'auto', 'ball_tree', 'kd_tree', 'brute'}, default='auto'

```
class sklearn.neighbors.KNeighborsClassifier(n_neighbors=5,  
*, weights='uniform', algorithm='auto', leaf_size=30, p=2,
```

```
metric='minkowski', metric_params=None, n_jobs=None)
[source]#
```

Parameters:

`n_neighbors`int, default=5

`weights`{‘uniform’, ‘distance’}, callable or None,
default=‘uniform’

`algorithm`{‘auto’, ‘ball_tree’, ‘kd_tree’, ‘brute’},
default=‘auto’

```
fit(X, y)[source]#
```

Parameters:

`X`{array-like, sparse matrix} of shape (n_samples,
n_features) or (n_samples, n_samples) if
`metric`=‘precomputed’

`y`{array-like, sparse matrix} of shape (n_samples,) or
(n_samples, n_outputs)

Returns:

Parameters

`metric_params`dict, default=None

Additional keyword arguments for the metric function.

`effective_metric_params_dict`

Additional keyword arguments for the metric function. For most metrics will be same with `metric_params` parameter, but may also contain the `p` parameter value if the `effective_metric_` attribute is set to 'minkowski'.

`X{array}`

Training data.

`y{array}`

Target values.

`paramsdict`

Parameter names mapped to their values.

`X{array}`

The query point or points. If not provided, neighbors of each indexed point are returned. In this case, the query point is not considered its own neighbor.

X {array}

The query point or points. If not provided, neighbors of each indexed point are returned. In this case, the query point is not considered its own neighbor. For `metric='precomputed'` the shape should be $(n_queries, n_indexed)$. Otherwise the shape should be $(n_queries, n_features)$.

A sparse

`n_samples_fit` is the number of samples in the fitted data. $A[i, j]$ gives the weight of the edge connecting i to j . The matrix is of CSR format.

X {array}

Test samples. If `None`, predictions for all indexed points are returned; in this case, points are not considered their own neighbors.

X {array}

Test samples. If `None`, predictions for all indexed points are returned; in this case, points are not considered their own neighbors.

`Xarray`

Test samples. If `None`, predictions for all indexed points are used; in this case, points are not considered their own neighbors. This means that `knn.fit(X, y).score(None, y)` implicitly performs a leave-one-out cross-validation procedure and is equivalent to `cross_val_score(knn, X, y, cv=LeaveOneOut())` but typically much faster.

`yarray`

True labels for `X`.

`sample_weightarray`

Sample weights.

`**paramsdict`

Estimator parameters.

Code Examples

```
>>> X = [[0], [1], [2], [3]]
>>> y = [0, 0, 1, 1]
>>> from sklearn.neighbors import KNeighborsClassifier
>>> neigh = KNeighborsClassifier(n_neighbors=3)
>>> neigh.fit(X, y)
KNeighborsClassifier(...)
>>> print(neigh.predict([[1.1]]))
[0]
>>> print(neigh.predict_proba([[0.9]]))
[[0.666 0.333]]
```

```
>>> samples = [[0., 0., 0.], [0., .5, 0.], [1., 1., .5]]
>>> from sklearn.neighbors import NearestNeighbors
>>> neigh = NearestNeighbors(n_neighbors=1)
>>> neigh.fit(samples)
NearestNeighbors(n_neighbors=1)
>>> print(neigh.kneighbors([[1., 1., 1.]])
(array([[0.5]]), array([[2]]))
```

```
>>> X = [[0., 1., 0.], [1., 0., 1.]]
>>> neigh.kneighbors(X, return_distance=False)
array([[1],
       [2]]...)
```

```
>>> X = [[0], [3], [1]]
>>> from sklearn.neighbors import NearestNeighbors
>>> neigh = NearestNeighbors(n_neighbors=2)
>>> neigh.fit(X)
NearestNeighbors(n_neighbors=2)
>>> A = neigh.kneighbors_graph(X)
>>> A.toarray()
array([[1., 0., 1.],
       [0., 1., 1.],
       [1., 0., 1.]])
```

Notes & Warnings

NOTE

See also RadiusNeighborsClassifierClassifier based on neighbors within a fixed radius. KNeighborsRegressorRegression based on k-nearest neighbors. RadiusNeighborsRegressorRegression based on neighbors within a fixed radius. NearestNeighborsUnsupervised learner for implementing neighbor searches.

⚠ WARNING

Warning Regarding the Nearest Neighbors algorithms, if it is found that two neighbors, neighbor $k+1$ and k , have identical distances but different labels, the results will depend on the ordering of the training data.

NOTE

See also NearestNeighbors.radius_neighbors_graphCompute the (weighted) graph of Neighbors for points in X.

Detailed Documentation

Documentation

Classifier implementing the k-nearest neighbors vote.

Read more in the User Guide.

Number of neighbors to use by default for kneighbors queries.

Weight function used in prediction. Possible values:

‘uniform’ : uniform weights. All points in each neighborhood are weighted equally.

‘distance’ : weight points by the inverse of their distance. in this case, closer neighbors of a query point will have a greater influence than neighbors which are further away.

[callable] : a user-defined function which accepts an array of distances, and returns an array of the same shape containing the weights.

Refer to the example entitled Nearest Neighbors Classification showing the impact of the weights parameter on the decision boundary.

Algorithm used to compute the nearest neighbors:

‘ball_tree’ will use BallTree

‘kd_tree’ will use KDTree

‘brute’ will use a brute-force search.

‘auto’ will attempt to decide the most appropriate algorithm based on the values passed

to fit method.

Note: fitting on sparse input will override the setting of this parameter, using brute force.

Leaf size passed to BallTree or KDTree. This can affect the speed of the construction and query, as well as the memory required to store the tree. The optimal value depends on the nature of the problem.

Power parameter for the Minkowski metric. When $p = 1$, this is equivalent to using `manhattan_distance` (l1), and `euclidean_distance` (l2) for $p = 2$. For arbitrary p , `minkowski_distance` (l_p) is used. This parameter is expected to be positive.

Metric to use for distance computation. Default is “minkowski”, which results in the standard Euclidean distance when $p = 2$. See the documentation of `scipy.spatial.distance` and the metrics listed in `distance_metrics` for valid metric values.

If metric is “precomputed”, X is assumed to be a distance matrix and must be square during fit. X may be a sparse graph, in which case only “nonzero” elements may be considered neighbors.

If metric is a callable function, it takes two arrays representing 1D vectors as inputs and must return one value indicating the distance between those vectors. This works for Scipy’s metrics, but is less efficient than passing the metric name as a string.

Additional keyword arguments for the metric function.

The number of parallel jobs to run for neighbors search. None means 1 unless in a `joblib.parallel_backend` context. -1 means using all processors. See Glossary for more details. Doesn’t affect fit method.

Class labels known to the classifier

The distance metric used. It will be same as the metric parameter or a synonym of it, e.g. 'euclidean' if the metric parameter set to 'minkowski' and p parameter set to 2.

Additional keyword arguments for the metric function. For most metrics will be same with metric_params parameter, but may also contain the p parameter value if the effective_metric_attribute is set to 'minkowski'.

Number of features seen during fit.

Added in version 0.24.

Names of features seen during fit. Defined only when X has feature names that are all strings.

Added in version 1.0.

Number of samples in the fitted data.

False when y's shape is (n_samples,) or (n_samples, 1) during fit otherwise True.

Classifier based on neighbors within a fixed radius.

Regression based on k-nearest neighbors.

Regression based on neighbors within a fixed radius.

Unsupervised learner for implementing neighbor searches.

See Nearest Neighbors in the online documentation for a discussion of the choice of algorithm and leaf_size.

Regarding the Nearest Neighbors algorithms, if it is found that two neighbors, neighbor $k+1$ and k , have identical distances but different labels, the results will depend on the

ordering of the training data.

https://en.wikipedia.org/wiki/K-nearest_neighbor_algorithm

Fit the k-nearest neighbors classifier from the training dataset.

The fitted k-nearest neighbors classifier.

Get metadata routing of this object.

Please check User Guide on how the routing mechanism works.

A MetadataRequest encapsulating routing information.

Get parameters for this estimator.

If True, will return the parameters for this estimator and contained subobjects that are estimators.

Parameter names mapped to their values.

Find the K-neighbors of a point.

Returns indices of and distances to the neighbors of each point.

The query point or points. If not provided, neighbors of each indexed point are returned. In this case, the query point is not considered its own neighbor.

Number of neighbors required for each sample. The default is the value passed to the constructor.

Whether or not to return the distances.

Array representing the lengths to points, only present if return_distance=True.

Indices of the nearest points in the population matrix.

In the following example, we construct a `NearestNeighbors` class from an array representing our data set and ask who's the closest point to `[1,1,1]`

As you can see, it returns `[[0.5]]`, and `[[2]]`, which means that the element is at distance 0.5 and is the third element of samples (indexes start at 0). You can also query for multiple points:

Compute the (weighted) graph of k-Neighbors for points in X.

The query point or points. If not provided, neighbors of each indexed point are returned. In this case, the query point is not considered its own neighbor. For `metric='precomputed'` the shape should be `(n_queries, n_indexed)`. Otherwise the shape should be `(n_queries, n_features)`.

Number of neighbors for each sample. The default is the value passed to the constructor.

Type of returned matrix: 'connectivity' will return the connectivity matrix with ones and zeros, in 'distance' the edges are distances between points, type of distance depends on the selected metric parameter in `NearestNeighbors` class.

`n_samples_fit` is the number of samples in the fitted data. `A[i, j]` gives the weight of the edge connecting `i` to `j`. The matrix is of CSR format.

Compute the (weighted) graph of Neighbors for points in X.

Predict the class labels for the provided data.

Test samples. If `None`, predictions for all indexed points are returned; in this case, points are not considered their own neighbors.

Class labels for each data sample.

Return probability estimates for the test data X.

Test samples. If None, predictions for all indexed points are returned; in this case, points are not considered their own neighbors.

The class probabilities of the input samples. Classes are ordered by lexicographic order.

Return the mean accuracy on the given test data and labels.

In multi-label classification, this is the subset accuracy which is a harsh metric since you require for each sample that each label set be correctly predicted.

Test samples. If None, predictions for all indexed points are used; in this case, points are not considered their own neighbors. This means that `knn.fit(X, y).score(None, y)` implicitly performs a leave-one-out cross-validation procedure and is equivalent to `cross_val_score(knn, X, y, cv=LeaveOneOut())` but typically much faster.

Mean accuracy of `self.predict(X)` w.r.t. y.

Set the parameters of this estimator.

The method works on simple estimators as well as on nested objects (such as Pipeline). The latter have parameters of the form `<component>__<parameter>` so that it's possible to update each component of a nested object.

Estimator parameters.

Configure whether metadata should be requested to be passed to the score method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with

`enable_metadata_routing=True` (see `sklearn.set_config`). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

True: metadata is requested, and passed to score if provided. The request is ignored if metadata is not provided.

False: metadata is not requested and the meta-estimator will not pass it to score.

None: metadata is not requested, and the meta-estimator will raise an error if the user provides it.

str: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Metadata routing for `sample_weight` parameter in `score`.

Classifier comparison

Caching nearest neighbors

Nearest Neighbors Classification

Comparing Nearest Neighbors with and without Neighborhood Components Analysis

Dimensionality Reduction with Neighborhood Components Analysis

Importance of Feature Scaling

Release Highlights for scikit-learn 0.24

Classification of text documents using sparse features

© Copyright 2007 - 2025, scikit-learn developers (BSD License).